

REVISED CAPSTONE READINESS REVIEW

AISERS System Audit Report

Requirements, implementation, security, permissions, database workflows, AI integration, paper consistency, and deployment readiness.

Project: NAAP–Villamor AISERS

Audit date: July 28, 2026

Role interpretation: OSA Administrator and System Administrator are combined into one account role, following the approved paper.

Scope: Application source, PHP APIs, JavaScript frontend, MySQL schema, project documentation, dependency audit, and available runtime checks.

Change policy: No application source files were modified during the audit.

1. Role Interpretation Used in This Report

Approved account roles: Student, Organization Officer, and combined OSA/System Administrator.

The paper treats OSA administration and system administration as responsibilities of the same authorized user. Therefore, this report does not classify the absence of a separate `system_admin` account as a missing feature. The existing `osa_staff` account type may represent the combined role, provided it includes the administrative capabilities stated in the paper.

The rule that a system administrator must not change OSA approval decisions is interpreted as a record-integrity rule: once the combined OSA/System Administrator finalizes an approval or rejection, the decision must not be silently overwritten. Corrections must use a new document version or a formally logged reconsideration process.

2. Executive Summary

Overall verdict: Not ready for production deployment. The combined OSA/System Administrator role is acceptable under the paper, but critical authorization, data-integrity, security, and workflow requirements remain incomplete.

The system's strongest areas are rental and service workflows, inventory, returns, fees and overtime, organization-scoped officer modules, dashboards, reports, and genuine AI-assisted descriptive analytics. The largest blockers are unsafe OSA/System Administrator account creation, weak authorization in account-management endpoints, public exposure of sensitive files, absent MFA and CSRF protection, no protected audit log, mutable document approvals, missing document versioning, and an attendance duplication defect.

Validation found 147 PHP files and 36 application JavaScript files free of syntax errors. Composer reported no known vulnerabilities in locked packages, and the existing OTP and transaction-email smoke tests passed. These results do not eliminate the application-level weaknesses below.

The active frontend is the HTML/CSS/JavaScript application backed by PHP and MySQL. The React/Vite directory is an unused prototype with placeholder routes and is not the active frontend.

3. Critical Issues

1. Broken administrative authorization. Several Accounts APIs call only `apiGuard()`, which verifies login but not the combined OSA/System Administrator role. A student or organization officer may be able to delete accounts or approve, reject, or reopen registrations.

`includes/auth.php:87; api/accounts/students/delete.php:6; api/accounts/requests/action.php:6`

2. Public OSA/System Administrator self-provisioning. OSA registration checks only whether an email and arbitrary employee number are unused. It does not require a staff whitelist, invitation, or approval by an existing authorized administrator.

`includes/otp.php:89; api/auth/register-osa.php:55`

3. Sensitive web-root exposure. Local HTTP testing confirmed that `capstone_db.sql` was downloadable and the uploads directory allowed listing. The dump contains user records and password hashes. Uploaded documents and technical files are also located under the public application tree.

4. Approved decisions are mutable. The combined administrator can repeatedly change a document between approved and rejected. Approved repository records are overwritten with `ON DUPLICATE KEY UPDATE`, and no immutable version history exists.
`includes/documents.php:249; includes/documents.php:279`

5. No protected audit log. There is no append-only `audit_logs` table recording the actor, action, target, organization, old state, new state, and timestamp. Dashboard activity assembled from mutable tables is not an audit log.

6. Event registration can duplicate attendance. Registration inserts an attendance row with a null check-in time. Barcode check-in searches only rows whose check-in date is today, misses that row, and inserts another. No event/student uniqueness constraint prevents duplication.
`api/student/events/register.php:58; includes/qr_attendance.php:492; capstone_db.sql:1120`

7. Essential session and request protections are absent. MFA, CSRF protection, Secure cookies, session-ID regeneration, timeouts, and sensitive-action reauthentication are missing or incomplete.

4. Requirement-to-Implementation Summary

Requirement	Status	Finding
Student	Implemented	Student accounts, dashboard, events, rentals, printing, lockers, and personal records exist.
Organization officer	Implemented	An officer is a student with an active organization membership role and dashboard access.
OSA/System Administrator	Partial/insecure	The existing <code>osa_staff</code> type can represent the combined role. Review and dashboard functions exist, but registration, MFA, account administration, audit oversight, and immutable decisions require work.
Registration, login, RBAC	Partial	Login and <code>bcrypt</code> exist; backend authorization is inconsistent.
Organization and membership management	Partial	Memberships and public profiles exist; complete organization create/update/archive management was not found.
Document upload, review, annotation	Implemented	Organization officers submit documents; OSA reviews and annotates them.
Document tracking	Partial	Basic pending/approved/rejected status exists, but the claimed SSC stage is absent.

Requirement	Status	Finding
Document versioning and archiving	Missing	No version table, parent link, version number, checksum, or archive lifecycle.
Announcements	Partial	Supports all students and specific courses, not the paper's Members Only and Officers audiences.
Role-targeted notifications	Partial	Transactional emails and computed student notices exist, but no general persistent notification/read-state model exists.
Events and registration	Partial	Implemented, but registration and attendance are stored together and can duplicate records.
Barcode attendance	Implemented	Code 128 student barcodes are generated through JsBarcode.
QR attendance	Inconsistent	The scanner can recognize QR codes, but student codes are generated as barcodes.
Rentals, services, inventory, fees	Strong	Reservations, inventory, returns, overtime, fees, printing, and lockers are substantially implemented.
Payment	Tracking only	Manual paid/unpaid recording correctly avoids out-of-scope online payment processing.
Dashboards, reports, analytics	Partial	All three approved role dashboards exist; compliance monitoring is mostly inferred client-side rather than stored as formal review records.
Audit logs	Missing	No protected append-only audit facility.
Centralized database	Implemented	MySQL supports operational modules, but required integrity tables are absent.
Responsive web interface	Substantially implemented	Viewport declarations and mobile CSS breakpoints exist. No native application is required.
User manual	Missing	No complete role-based end-user guide was found.

5. Role and Permission Findings

- **Student:** Should access student events, rentals, printing, lockers, notifications, and personal records only.
- **Organization Officer:** Should remain a student account with active organization membership and organization-scoped authority.
- **OSA/System Administrator:** May combine document review, compliance, account management, organization administration, configuration, audit-log viewing, and technical oversight.
- The combined administrator account must not be publicly self-registered. Initial accounts should be seeded securely; additional accounts should require verified staff invitation or approval.
- Organization ownership checks are generally strong in document, announcement, rental, and attendance helpers, but Accounts endpoints fail to enforce the administrator role consistently.
- Students do not submit organizational approval documents. Their upload area is for printing-service requests, which matches the context diagram's division of responsibilities.

- Guests and renters are not separate account roles. Renters are students, while unauthenticated people are visitors to login and registration pages.

6. Approval Integrity for the Combined Administrator

Combining OSA and system-administration responsibilities does not justify changing finalized approval records. The required control should operate as follows:

1. A pending submission may be reviewed and approved or rejected by an authorized OSA/System Administrator.
2. The finalized decision and approved file become immutable.
3. A correction or revised submission creates a new linked document version.
4. If reconsideration is allowed, it must be a separate workflow with a mandatory reason and append-only audit entry.
5. Account-management or system-configuration functions must never directly update finalized approval rows.

7. Database and Workflow Mismatches

- No `audit_logs`, `document_versions`, `general_notifications`, or separate `event_registrations` table exists.
- Document workflow uses pending, approved, and rejected rather than Pending → SSC Review → OSA Review → Approved/Rejected.
- Approving and later rejecting a document can leave the approved-repository row present.
- Event registration and actual participation are conflated in `attendance_records`.
- The deployed rental schema tracks payment status and paid date but not the `payment_method` described in the design paper.
- The design paper describes version fields and different inventory quantities, while deployed code uses database transactions and individually tracked inventory records.
- Organization/event deletion behavior can conflict with long-term participation and audit-record retention.

8. Security Assessment

Control	Status	Assessment
Argon2id or bcrypt	Pass	Bcrypt is used for account passwords.
MFA for combined administrator	Fail	Email registration/reset OTP is not login MFA.
Secure sessions	Partial	HTTP-only and SameSite=Lax exist; Secure flag, ID regeneration, inactivity timeout, absolute lifetime, and sensitive-action reauthentication are absent.
CSRF protection	Fail	No CSRF-token system was found for cookie-authenticated mutations.
Server authorization	Inconsistent	Strong in several domain helpers; critically weak in Accounts APIs.

Control	Status	Assessment
Organization ownership	Mostly pass	Core officer modules normally verify active organization context and scope SQL by organization.
HTTPS/TLS 1.3	Unverified	Local service used HTTP. Production TLS must be configured and verified on the cloud host.
Prepared SQL	Mostly pass	PDO prepared statements are widespread and emulated prepares are disabled.
Input/file validation	Partial	Document PDFs are checked by filename extension only, without MIME/signature or size validation.
Encryption at rest	No evidence	No database, document, dump, or backup encryption implementation was found.
Protected audit logs	Fail	No audit log exists.
No hard-coded secrets	Partial	AI/mail use environment secrets, but database configuration uses MySQL root with a blank password.
Immutable approvals	Fail	Final decisions and approved repository records can currently be overwritten.

9. AI Assessment

Real AI-assisted analytics exists; ordinary totals and charts were not counted as AI.

- **Primary provider:** Google Gemini.
- **Models:** `gemini-2.5-flash` and `gemini-2.5-flash-lite`.
- **Local fallback:** Ollama using `llama3.2:3b`.
- **Final fallback:** Deterministic rule-based summaries.
- **Purpose:** Descriptive narratives for revenue, participation, rentals/inventory, and document-workflow aggregates—not prediction or machine learning.
- **Data sent:** Filtered totals, chart series, summaries, and limited event details. The current builder does not send student names, emails, or student numbers.
- **API-key handling:** The Gemini key is read from a server environment variable rather than hard-coded source.

10. Paper and Implementation Contradictions

1. One feature document calls the system deployment-ready while architecture documents label OSA, documents, and notifications partial or planned.
2. Terminology should consistently use “OSA/System Administrator” if the approved paper combines the responsibilities.
3. The paper correctly assigns organizational document submission to officers and rental/service requests to students.
4. The claimed SSC review stage is not implemented.
5. Claimed announcement audiences differ from the implemented course-targeted model.

6. "QR Attendance" conflicts with actual Code 128 barcode generation.
7. Offline attendance, external calendar synchronization, financial audit records, policy updates, and formal compliance records are claimed without complete persistent workflows.
8. The proposed ERD contains fields and tables not present in the deployed schema.
9. Main dashboards are static HTML shells using PHP JSON APIs, so the architecture is hybrid monolithic but only partly server-rendered.
10. Payment functionality is manual tracking, not online payment processing.

11. Prioritized Fixes Before Defense

P0 Immediate blockers

1. Restrict every Accounts endpoint to the combined OSA/System Administrator role.
2. Disable public administrator registration; use securely seeded accounts and verified staff invitations.
3. Remove SQL dumps, technical documents, backups, and raw uploads from public access; disable directory indexes.
4. Make finalized OSA decisions immutable and require a new version or logged reconsideration for corrections.
5. Add CSRF protection, Secure cookies, session regeneration, timeouts, and sensitive-action reauthentication.
6. Repair registration/check-in duplication and add database uniqueness protection.

P1 Required integrity and administrator features

7. Add append-only protected audit logs accessible to the combined administrator.
8. Add immutable document versions and archiving.
9. Separate event registrations from participation and attendance.
10. Add persistent role-targeted notifications with read and delivery states.
11. Implement MFA for all OSA/System Administrator accounts.
12. Complete administrator capabilities for account, organization, configuration, and audit oversight without allowing direct alteration of finalized approvals.

P2 Deployment hardening

13. Validate PDF MIME type, signature, size, and authorized download access.
14. Use environment secrets and a least-privilege production database account.
15. Configure and verify TLS 1.3, HSTS, database/document/backup encryption at rest, and secure backups.
16. Add centralized error handling and security headers.
17. Replace runtime schema changes with controlled database migrations.

P3 Defense alignment and quality

18. Use "OSA/System Administrator" consistently in the interface, manual, diagrams, and defense explanation.
19. Implement remaining paper-claimed workflows, including SSC review and required notification audiences.
20. Describe payments as manual tracking and AI as assisted descriptive analytics.
21. Either implement genuine QR generation or consistently document barcode attendance.
22. Remove the unused React/Vite prototype and mock-data controls from the production package.
23. Create a role-specific user manual and documented responsive-browser test results.
24. Repair broken navigation and hard-coded local deployment paths.

12. Defense Position

The defensible role explanation is:

"OSA personnel also perform authorized system-administration functions. OSA Administrator and System Administrator are functional responsibilities under one secured account role, not separate user types. Finalized OSA decisions remain immutable, and every sensitive administrative action is audited."

This combined-role design is acceptable if it is used consistently and the missing security, administrative, audit, and record-integrity controls are implemented.

AISERS Revised Capstone System Audit Report · Combined OSA/System Administrator role interpretation.